

HN.FM · PIPELINE ARCHITECTURE

From a Hacker News link to an **AI-directed video**

A complete walk through hn.fm's generation pipeline — the inference services, the Celery orchestration, the prompts, and the agentic "director" that decides how each beat of a story is told. Followed end-to-end with one real generation.

HN Firebase API

Firecrawl

Nemotron (LLM + Vision + ASR)

FLUX.2 Klein

Magpie TTS

ACE-Step

LTX-2

HyperFrames

8

inference services, one Celery
worker

7

pipeline stages, strictly linear

3

ways to tell a section: sequence
· motion · kinetic text

The big picture

A story ID enters; a finished MP4 leaves. Everything runs in **one Celery worker, one stage at a time, one service call at a time** — state lives in Redis, artifacts on disk. No step runs in parallel with another; this keeps a shared GPU cluster sane and the process easy to reason about.

0 Fetch `hn_fetch_item`

HN Firebase API → store the item (title, URL) in Redis.



1 Scrape & summarize `process_hn_item_run`

Firecrawl **Nemotron LLM** article → clean text → **summary, tags, emoji, haiku.**



1b Source images `ingest_source_images`

Firecrawl HTML **Nemotron Vision** pull real article images → rank by alt → downscale → describe.



2 Script `generate_segment`

Nemotron LLM two-host [S1] / [S2] podcast dialogue from the summary + article.



3 Audio `build_segment_audio`

Magpie TTS **Nemotron ASR** speak each line (2 voices) → transcribe for word timings → QA vs script.



4 Images & sequences `build_segment_images`

Nemotron LLM **FLUX.2 Klein** per section: write a cinematic prompt → render a root image → image-to-image edit it into a short evolving sequence.



4b The director `build_segment_media_plan`

Nemotron LLM **LTX-2** **HyperFrames** plan a **template per section** (sequence / motion clip / kinetic-text) → produce the special clips.



5 Assemble `generate_segment_video`

ACE-Step **ffmpeg** build the timeline → burn animated subtitles → mix a music bed → encode the MP4.

The inference services

All models run on a home Kubernetes (k3s) cluster as cluster-internal services, reached from the local stack over tunnels. One LLM family — **Nemotron** — does triple duty (text, vision, speech-to-text).

SERVICE	ROLE IN THE PIPELINE	INTERFACE
HN Firebase	Source of stories (id → title, url)	REST
Firecrawl	Scrape the article to markdown + pull its images	/v1/scrape
Nemotron LLM	Summary, script, image prompts, the director's plan	OpenAI /v1 (LiteLLM)
Nemotron Vision	Describe real source images; spot-check frames	OpenAI /v1 (image_url)
Nemotron ASR	Word-level timestamps for animated subtitles + QA	/v1/audio/transcriptions
FLUX.2 Klein	Text-to-image (root) & image-to-image (sequence edits)	/v1/images/generations · /edits
Magpie TTS	Two-voice narration ([S1] Mia / [S2] Jason)	/v1/audio/synthesize
ACE-Step	Instrumental music bed (theme-matched)	/v1/chat/completions (audio)
LTX-2	Image-to-video motion clips (hero moments)	/generate
HyperFrames	HTML+GSAP → MP4 kinetic-text clips (render sidcar)	/render (Node+Chromium)

Orchestration model

One worker, linear

A single Celery worker (concurrency=1) runs `full_pipeline` — which calls each stage in sequence, not as async sub-tasks. Within a stage, calls are sequential too (e.g. images render one section at a time). So the cluster only ever serves one request at a time.

Redis is the state

No SQL. Keys hold the run, the segment (script, plan, flags), per-section sections & images, source images, and a `metrics` record. Artifacts (audio, images, clips, the MP4) live on a shared outputs/volume.

Why everything is non-fatal

A shared GPU can hiccup. Every optional service (source images, LTX, HyperFrames, music) is wrapped so a failure degrades gracefully — a missing motion clip just falls back to the image sequence — and the video still finishes.

THE PROMPTS

What each model is asked

Prompts are summarized. The pattern throughout: a tight **system role** + the story context, asking for one specific, structured thing.

① Summary & metadata — Nemotron LLM

```
# role Summarize this article for a tech-news short.  
Return a crisp summary, a 1-2 sentence hook, 2-6 tags, 4 emoji, and a haiku.  
# input {cleaned article text}
```

② Script — Nemotron LLM → generate_script_v1

```
# role Write a two-host podcast script about this story.  
Alternate speakers tagged [S1] and [S2]; natural, punchy, accurate.  
One idea per line; conversational; no stage directions.  
# input {summary} + {article}
```

③ Image prompt (per section) — Nemotron LLM → generate_image_prompt_v1

```
# role You are a cinematic art director writing ONE image prompt.  
Invent a striking, SPECIFIC scene: a clear subject doing something,  
a concrete setting, evocative props, a deliberate shot type.  
Describe ONLY scene + composition — do NOT mention art style.  
# then (deterministic) append the take's THEME style block  
    so every shot in the video is stylistically cohesive.
```

④ The director's plan — Nemotron LLM → meta_sequencer

```
# role You are the director. For each narration section choose:  
    image_sequence (default) · video (a hero motion moment) ·  
    hyperframe (structured kinetic text: keypoints/bigstat/quote/compare).  
# taste interesting, educational, funny, unexpected, original.  
    most sections stay sequences; sprinkle ≤2 video & ≤2 hyperframe;  
    never two specials back-to-back.  
# return JSON: one {template, recipe?, content?} per section.
```

A deterministic **guardrail** pass then enforces the caps/spacing, and a second LLM **critic** scores the plan on the taste rubric and revises weak spots.

⑤ Vision & ASR — Nemotron

```
# vision Describe this image in 1-2 sentences: subjects, mood.  
# asr Transcribe with word-level timestamps (for karaoke subtitles).
```

The director: how a section becomes a shot

Stage 4b is where hn.fm stops being a fixed template and starts making choices. It runs **plan** → **guardrails** → **critic** → **guardrails**, then produces only the special clips.

1 • Plan

LLM reads all sections + summary + theme + available source images, assigns a template to each.

→

2 • Guardrails

Deterministic: caps (≤ 2 each), no two specials adjacent, sequence is the floor. Fixes violations.

→

3 • Critic

A second LLM scores interest/originality, flags repetition, suggests swaps. Guardrails run again.

The three ways to tell a section



image_sequence

The default. A root image, then FLUX **image-to-image edits** that evolve it — shown in quick cadence. Most sections.



video

An **LTX-2** motion clip from the section's still + a motion prompt, lightly stretched. A hero moment.



hyperframe

HyperFrames kinetic text (keypoints / bigstat / quote / compare) over the image — for structured beats.

Every special clip is folded into the timeline as a `video_clip_path` with **partial coverage**: it plays its natural length, then the image sequence fills the rest of the section — so no downstream step needs to know which kind it was.

"These Are the Most Beautiful Equations"

Source: [scientificamerican.com](https://www.scientificamerican.com) · theme **Technical Blueprint** · 16:9 · 21 sections. Tags

geometry



① Summary (Nemotron)

Mathematicians view equations as artistic expressions of beauty — valuing simplicity, unexpected connections, and profound insights. The Loewner equation links complex analysis to statistical mechanics; Gauss-Bonnet ties curvature to topology...

② Script (two-host dialogue)

[S1] Welcome to Proof Positive, your science podcast.
 [S2] Today we're exploring the most beautiful equations mathematicians love.
 [S1] To them, equations are like art — simple, surprising, or deeply insightful.
 [S2] Let's start with the Loewner differential equation...

③ A real image prompt (§1, before the theme block)

A mathematician in a sunlit study stands before a massive chalkboard, meticulously drawing a single elegant equation that weaves together the Gauss-Bonnet formula and the Euler product, with a vintage compass...
 → + "Technical Blueprint" style appended deterministically

④ The director's plan (21 sections)



§1 **SEQ** · §2 **HYPER** · §3-9 **SEQ** · §10 **VIDEO** · §11-18 **SEQ** · §19 **VIDEO** · §20-21 **SEQ**

→ 18 sequences, 2 motion clips, 1 hyperframe. The hyperframe content the LLM wrote for §2:

keypoints · kicker "Math is art" · title "The Art of Equations"

- Simple beauty in complex forms
- Surprising connections across fields
- Deep insights in minimal expressions

⑤ Real source images (Nemotron-described, from the article)

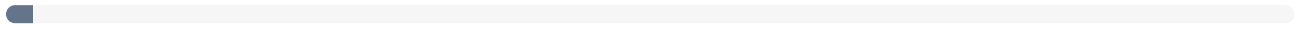
4 chalkboard photos pulled & described, e.g. *"a complex mathematical formula for the partition function $p(n)$ written in white chalk on a dark blackboard."*

Where the time & tokens go

Measured per stage by the built-in metrics (visible live on the /observability dashboard). One ~3-minute, 19-section video:

Total render

429s (~7 min) · ~13k tokens



■ scrape 9s ■ source 0.2s ■ script 10s ■ audio 21s ■ images 220s ■ media_plan 79s ■ assemble 90s

51%

images — the bottleneck. ~57 sequential FLUX calls (19 sections × root + edits).

21%

assemble — ffmpeg concat of ~30 timeline segments + subtitle burn + music mix.

18%

media_plan — mostly one LTX clip (~70s) + plan & critic LLM calls.

Levers

Fewer sequence frames (SEQ_MAX_FRAMES) cuts the 51% chunk most. Tokens concentrate in image-prompt generation. LTX reliability (a shared GPU) is the main reason a planned *video* sometimes drops to a sequence.

How this document was made

So we can regenerate it as the pipeline evolves.

1. Pulled the prompt templates from source (`segment_utils`, `meta_sequencer`) and summarized them.
2. Pulled a **real recent generation** from Redis (script, image prompts, the director's plan, hyperframe content, source-image descriptions).
3. Pulled **real stage timings & token counts** from the metrics record (`/api/metrics`).
4. Authored this as a single on-brand **HTML** file (`pipeline-guide/pipeline.html`) — diagrams are CSS/flex, no external libs.
5. Rendered to **PDF** and **social PNGs** with headless Chromium (the HyperFrames render sidecar): `chromium --headless --print-to-pdf / --screenshot`.

Artifacts live in `pipeline-guide/`: `pipeline.pdf`, `pipeline.html`, and `assets/*.png` (square + wide social cards). Re-run: edit the HTML, re-render.

hn.fm — turning the front page into watchable stories.